

Smart Lender - Loan Eligibility Prediction System

This documentation summarizes the AIML project.

Smart Lender - Loan Eligibility Prediction System

Smart Lender is an end-to-end machine learning project that predicts loan eligibility using a production-ready Flask web application. The system combines data preprocessing, exploratory data analysis, feature engineering, model comparison, hyperparameter tuning, and a modern responsive UI.

Project Overview

This project is designed to evaluate loan applicants using a trained classification pipeline. It compares four machine learning models, improves them with cross validation and GridSearchCV, and saves the best-performing tuned model for inference.

The system includes:

- Data preprocessing and missing-value handling
- Exploratory Data Analysis notebooks
- Feature engineering and shared input schema
- Model training with Decision Tree, Random Forest, KNN, and XGBoost
- Cross-validation and hyperparameter tuning
- Feature selection and optional SMOTE for class imbalance
- Flask-based prediction web app
- Responsive UI with Bootstrap 5
- Deployment-ready project structure

Features

- Modern banking-style homepage
- Loan prediction form with client-side validation
- Prediction result page with approval probability and risk level
- Baseline vs tuned model comparison
- Matplotlib comparison charts
- Best model saved using Joblib
- Production-friendly WSGI entrypoint
- Error handling and validation
- Clean modular code

Technologies

Backend

- Python 3.11
- Flask
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Joblib
- imbalanced-learn

Data Visualization

- Matplotlib

- Seaborn

Frontend

- HTML5
- CSS3
- Bootstrap 5
- JavaScript

Installation

1. Clone the repository

```
```bash
git clone
cd Smart-Lender
```
```

2. Create a virtual environment

```
```bash
python -m venv .venv
```
```

3. Activate the virtual environment

On Windows:

```
```bash
.venv\Scripts\activate
```
```

On macOS/Linux:

```
```bash
source .venv/bin/activate
```
```

4. Install dependencies

```
```bash
pip install -r requirements.txt
```
```

How to Run

1. Add the dataset

Place the loan dataset in:

```
```text
dataset/loan_data.csv
```
```

The project expects a loan dataset with the standard fields used in the form and preprocessing pipeline.

2. Train and tune the model

```
```bash
python model/train_model.py
```
```

This script will:

- Load and clean the dataset
- Run baseline model evaluation
- Apply cross validation
- Tune hyperparameters with GridSearchCV
- Use feature selection
- Apply SMOTE when the dataset is imbalanced
- Compare before and after results
- Save the best pipeline as `model/model.pkl`
- Save the comparison table and summary artifacts
- Generate a Matplotlib comparison chart

3. Start the Flask app

```
```bash
python app.py
```
```

4. Open the application

Visit the local server in your browser:

```
```bash
http://127.0.0.1:5000
```
```

Screenshots

Add your screenshots to `static/images/` and reference them here.

```
```markdown



```
```

Project Structure

```
```text
Smart-Lender/
■■■ app.py
■■■ wsgi.py
■■■ Procfile
■■■ runtime.txt
■■■ gunicorn.conf.py
■■■ requirements.txt
■■■ README.md
```
```

```
■■■ DEPLOYMENT.md
■■■ dataset/
■ ■■■ loan_data.csv
■■■ model/
■ ■■■ preprocessing.py
■ ■■■ train_model.py
■ ■■■ predict.py
■ ■■■ model.pkl
■ ■■■ training_results.csv
■ ■■■ training_summary.json
■ ■■■ plots/
■ ■■■ model_comparison.png
■■■ templates/
■ ■■■ index.html
■ ■■■ predict.html
■ ■■■ result.html
■■■ static/
■ ■■■ css/
■ ■ ■■■ style.css
■ ■■■ js/
■ ■ ■■■ script.js
■ ■■■ images/
■■■ notebooks/
■■■ EDA.ipynb
...

```

Machine Learning Workflow

1. Load the loan dataset
2. Inspect the data and perform EDA
3. Clean missing values
4. Build a shared feature schema
5. Split features and target
6. Encode the target variable
7. Split the data into train and test sets
8. Build a preprocessing pipeline with scaling and one-hot encoding
9. Evaluate baseline models
10. Tune Decision Tree, Random Forest, KNN, and XGBoost with GridSearchCV
11. Apply feature selection with SelectKBest
12. Use SMOTE when class imbalance is detected
13. Compare baseline and tuned results
14. Select the best pipeline using both performance and generalization gap
15. Save the best model with Joblib
16. Use the trained pipeline in Flask for live predictions

Future Enhancements

- Add authentication and user roles
- Store prediction history in a database
- Add SHAP explainability to show feature impact
- Build an admin analytics dashboard
- Add API endpoints for external integrations
- Containerize the application with Docker
- Improve deployment automation with CI/CD

Author

Mandapaka Pushpavalli